

Inference of Protein Secondary Structure with HMMs and BayesFlow

Simulation-Based Inference — Project Report

Armin Maddah Asl Samaneh Naghdbishi Kiana Ebrahimi Farbod Geri Lila

July 15, 2026

TU Dortmund University

Course Instructor: Prof. Dr. Paul Bürkner

Introduction & Modeling Idea

The Problem

Predict per-position secondary structure
from variable-length amino acid sequences.

Two-State HMM

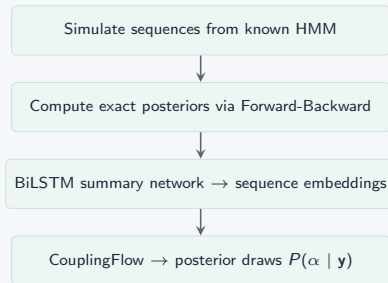
State 0 = α -helix

State 1 = other (β -sheets & coils)

Modeling Goal

Approximate $P(S_t = \alpha \mid \mathbf{y})$ at every position t
using **BayesFlow** neural posterior estimation.

Overall Pipeline



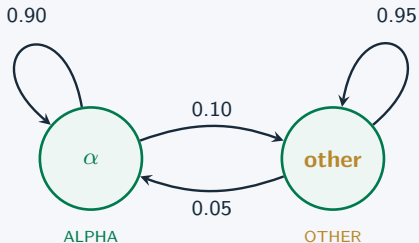
Validation

Compare to experimental ground truth:

Human insulin **1A7F** (chain A: 21 aa, chain B: 29 aa).

The 2-State Hidden Markov Model

State Transitions



Deterministic start: $P(S_1 = \text{other}) = 1.0$, $P(S_1 = \alpha) = 0.0$

Highly persistent: α stays α (90%), other stays other (95%)

Selected Emission Contrasts

Amino acid	α -helix	other	Role
A (Alanine)	12%	6%	Strong former
E (Glutamic)	9%	5%	Former
P (Proline)	2%	6%	Breaker
G (Glycine)	4%	9%	Breaker

Full 20×2 table in written report.

Prior Dynamics

Stationary distribution:

$$\pi(\alpha) = 0.05 / (0.05 + 0.10) = 1/3 \approx 33\%$$

$$\pi(\text{other}) \approx 67\%$$

Prior strongly favors **non-helical** states ($\sim 67\%$).

Data Generation & Analytical Targets

Simulator

Sequential Markov sampling of hidden states.

Independent randomized emission draws.

Random sequence lengths: 10–60 aa.

100% simulated — no real proteins.

Target Computation

Exact posteriors via Forward-Backward.

`hmmlearn.CategoricalHMM` — manually

configured, never `.fit()`.

Verification

Targets verified against independent

pure-NumPy FB solver. Confirms:

$P(S_0 = \alpha) \equiv 0$ (position 0 = seq. position 1).

On-demand Generation

`generate_dataset(n, ...)` produces

any number of (sequence, posterior, states) tuples. Downstream code

chooses the split sizes.

Variable-Length by Design

Sequences kept **ragged** (10–60 aa),
each stored with its own length.

The data layer does **not pad** — padding
& masking are handled downstream by

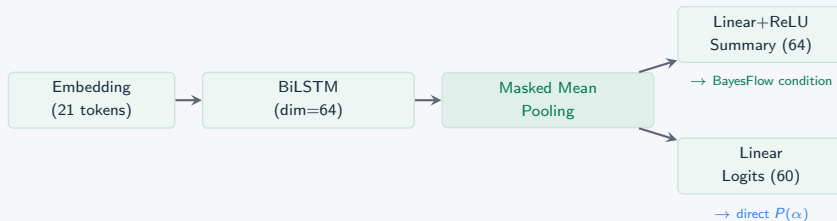
the summary network.

Storage

`.npz` format: concatenated arrays +

`lengths` vector. No object arrays.

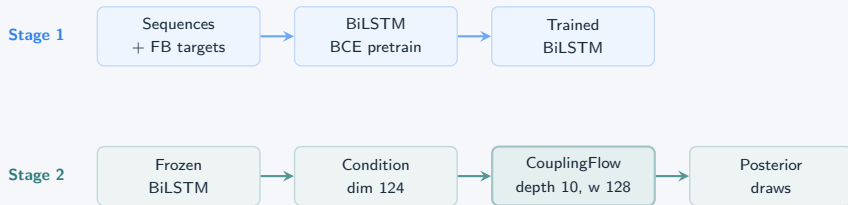
The Summary Network — BiLSTM



1. BiLSTM captures **sequential** dependencies — permutation-invariant nets fail here.
2. Padding token 20 & masked loss → handles variable-length sequences.
3. Two-stage: BiLSTM pretrained on BCE → frozen → CouplingFlow on top.

Two-Stage Training & CouplingFlow

Training Pipeline



Key Design Decisions

1. BiLSTM pretrained on masked BCE, then **frozen** → condition (dim 124 = 64 pooled + 60 logits).
2. CouplingFlow: depth 10, hidden 128, affine, actnorm → posterior draws.
3. **Model selection:** not flow val_loss → rank by masked MAE & correlation (selected depth 10).

Diagnostics: Adapted Simulation-Based Calibration

Standard SBC

Draw $\theta \sim p(\theta)$ from prior.

Simulate data $\mathbf{x} \sim p(\mathbf{x} \mid \theta)$.

Compute posterior, check if ranks of θ_{true} are uniform.

Our Challenge

HMM parameters are **fixed & known**.

$P(\alpha_t \mid \mathbf{y})$ is a **deterministic** function of the input — no parameter uncertainty to recover.

Our Adapted Method

1. Draw S test sequences from the HMM.



2. Generate M posterior draws from CouplingFlow at each position t .



3. Rank the true FB target value within the flow's posterior draws.



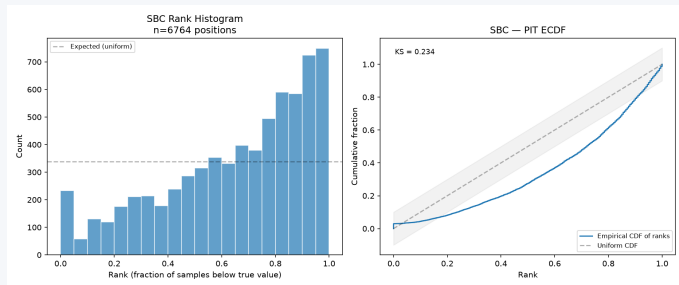
4. Test uniformity: rank histogram, PIT ECDF, KS statistic.

What This Tests

Does the BayesFlow posterior **envelope** the deterministic FB value with correct frequency?

Note: Not a classical prior-predictive SBC — adapted for the deterministic-target setting.

Calibration Results — Adapted SBC



Systematic Underestimation

Rank histogram shifted **rightward**:

mean rank = **0.65**

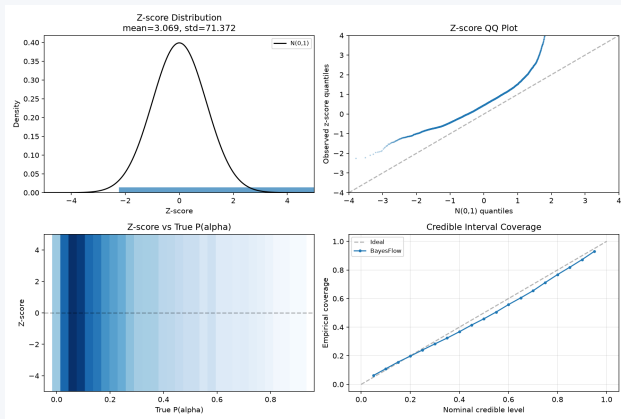
only **27%** below 0.50

KS = 0.234

True FB $P(\alpha)$ is frequently **larger** than posterior draws.

→ Padded zero boundaries pull the continuous flow toward 0.

Calibration Results — Z-Score & Coverage



Undercoverage & Bias

Raw z-score std = **71.4**

Trimmed (1.1% outliers removed):
mean = **0.63**, std = **1.27**

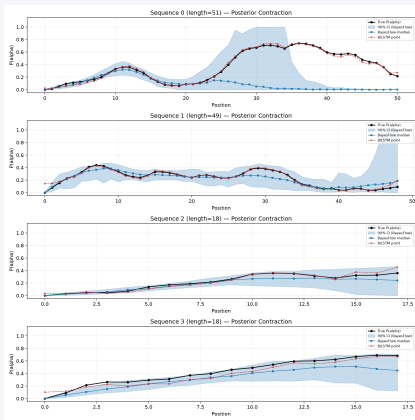
87% empirical vs. 90% nominal

93% empirical vs. 95% nominal

Posteriors are **too narrow**
and **slightly biased**.

→ Localized overconfidence
at sequence boundaries.

Synthetic Evaluation — Posterior Contraction



Black = true $P(\alpha)$. Blue band = 90% CI. Red = BiLSTM point estimate.

BiLSTM Wins on Accuracy

	BayesFlow	BiLSTM
MAE	0.115	0.021
Corr	0.560	0.988

FB target is **deterministic**
— direct network learns it
more efficiently than a
density estimator.

But: BayesFlow gives
uncertainty quantification
— credible intervals,
SBC, z-scores, coverage.

Real-World Validation — Human Insulin (1A7F)

Chain B

BayesFlow

$r = 0.86$

34.5% true helix

$\approx \pi = 33\%$

Chain A

BayesFlow

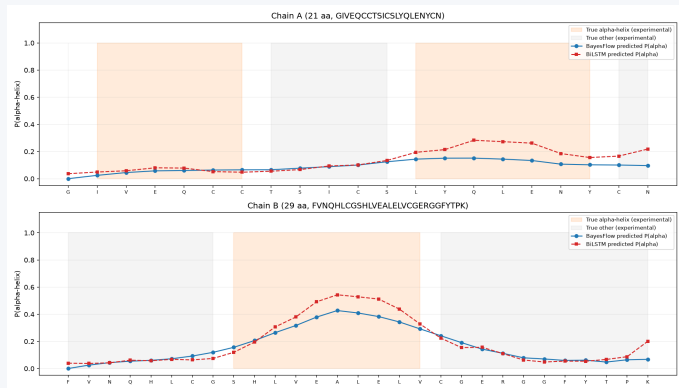
$r = 0.17$

61.9% true helix

$\gg \pi = 33\%$

Key proof: FB itself
fails on Chain A
($r \approx 0.40$).

→ **Model limitation,**
not network failure.



Orange = experimental α -helix. Blue = BayesFlow. Red = BiLSTM.

Limitations, Fixes & Take-Home Messages

Limitation & Fix: The HMM

Expand to **3 states** (α , β , coil).

Treat transition probabilities as **inferable** parameters θ under Beta priors.

→ BayesFlow would then recover a genuine posterior over θ .

Fix: The CouplingFlow

Pass targets through a **logit transformation**

before training → strictly bound all draws in $(0, 1)$.

Take-Home Messages (TL;DR)

1. BiLSTM summary nets capture sequential dependencies where permutation-invariant nets fail.
2. BayesFlow successfully amortizes posterior estimation, enabling robust uncertainty quantification.
3. Real-world validation demonstrates the necessity of prior typicality checking — identify prior-data conflicts.

Group Members

Armin Maddah Asl

armin.maddah@tu-dortmund.de

Samaneh Naghdbishi

samaneh.naghdbishi@tu-dortmund.de

Kiana Ebrahimi Farbod

kiana.ebrahimi@tu-dortmund.de

Geri Lila

geri.lila@tu-dortmund.de

Code:

github.com/armin2080/

[protein-secondary-structure-npe](#)